智能机器人系统综合实验

# 基于视觉与地图导航的多机协同控制

方案设计与实验报告（协作初稿）


课程/班级： 待填写课程与班级

团队名称： 第一组（人形机器人与方式二）及协作小组

成员姓名、学号： 见第2节分工表



报告日期： 2026 年 10 月 15 日

代码依据： `6b6db8c`



本报告汇总 TonyPi 抱物穿行与四类 TurtleBot2 跟随方案。本次重点撰写人形机器人、方式二建图避障跟随；其余方式由对应小组在独立源文件中补充。姓名、个人贡献与定量实验数据尚待各组核实。文中区分代码实现、离线验证、局部实机验证与完整比赛验收。

# 摘要

本实验以一台 TonyPi 人形机器人和多台 TurtleBot2 为平台，研究抱物穿行、视觉航路控制及多方式跟随的系统集成。任务要求人形机器人抱持物体完成至少一次左转和一次右转，终点放置物体；各移动机器人以可区分的方式保持跟随，并满足避碰、目标保持和终点停靠要求。

人形机器人部分采用“动作组执行＋视觉反馈”的分层结构：以抓取末帧固定上肢姿态，复用下肢步态；先通过开环路线定位动作偏移问题，再实现灰色挡板道路居中，最新版本进一步采用 A/B/C 固定标记进行测距、对中与转角切换。方式二采用赛前 GMapping 建图、比赛 AMCL 定位、AR 观测变换至地图、历史轨迹回退生成目标以及 move_base/DWA 导航，并设置独立速度门控。报告给出分工模板、接口约定、实现原理、已知验证范围和实验记录表，供各组并行补充。

现有仓库记录支持局部实机运行与部分软件验证，尚不足以证明整条路线及全队比赛验收完成。本文不将仿真结果、默认参数或设计目标作为实测成绩。

**关键词：** TonyPi；TurtleBot2；动作组；视觉标记；地图导航；多机协同

# 目录

**摘要** **I**

**1 实验任务与场地** **1**

1.1 任务要求与验收口径 . . . . . 1

1.2 场地与感知约束 . . . . . 1

**2 团队组织与成员分工** **2**

2.1 小组之间的职责 . . . . . 2

2.2 本小组内部职责登记 . . . . . 3

2.3 其他小组内部职责登记规范 . . . . . 3

**3 系统架构与协同接口** **4**

3.1 分层控制 . . . . . 4

3.2 软件平台与版本边界 . . . . . 4

3.3 拓扑与跟随方式的区分 . . . . . 4

**4 第一组：TonyPi 人形机器人抱物与视觉路线控制** **5**

4.1 方案演进与入口区分 . . . . . 5
4.2 动作组格式与抱物步态组合 . . . . . 5
4.3 抓取、持物观察与头部姿态 . . . . . 7
4.4 早期灰色挡板道路居中 . . . . . 7
4.5 最新 A/B/C 固定标记识别与测距 . . . . . 7
4.6 完整路线状态机与自适应纠偏 . . . . . 8
4.7 相机可靠性与可视化调试 . . . . . 8
4.8 已有验证与后续验收 . . . . . 9

**5 第二组（暂定）：方式一 AR 直接跟随 9**
5.1 已有实现摘要与方式认定 . . . . . 9
5.2 本组应补充的内容 . . . . . 10

**6 第一组：方式二建图避障跟随 10**
6.1 方案与功能包组织 . . . . . 10
6.2 沿用课程建图链与地图质量检查 . . . . . 10
6.3 AR 目标观测到地图坐标 . . . . . 11
6.4 历史轨迹与沿路径回退的跟随点 . . . . . 11
6.5 代价地图、DWA 与目标可行性 . . . . . 12
6.6 双层许可与速度门控 . . . . . 12
6.7 任务状态联动与当前集成缺口 . . . . . 13
6.8 验证范围与待测指标 . . . . . 13

**7 第三组（暂定）：方式三 RGB-D 实时检测与避障跟随 13**
7.1 已有实现摘要 . . . . . 13
7.2 本组补充区域 . . . . . 14

**8 方式四：TurtleBot2 视觉跟随与队列行驶 14**
8.1 方案与队列关系 . . . . . 14
8.2 系统组成与控制方法 . . . . . 14
8.3 参数设置及含义 . . . . . 15
8.4 执行流程与运行参数 . . . . . 16
8.5 实验记录 . . . . . 17

8.6 小组分工 . . . . . 18
8.7 小结 . . . . . 18

**9 实验设计、证据与结果记录 18**
9.1 验证层级与当前事实 . . . . . 18
9.2 分阶段实验与异常注入 . . . . . 19
9.3 指标定义 . . . . . 19
9.4 数据归档与照片使用 . . . . . 20

**10 问题分析与总结 20**

**参考资料与实现依据 21**

**A 复现入口与协作清单 21**
A.1 源码入口索引 . . . . . 21
A.2 只查看最新标记检测 . . . . . 22
A.3 方式二现场启动索引 . . . . . 22
A.4 报告提交前核对 . . . . . 23

# 1 实验任务与场地

## 1.1 任务要求与验收口径

依据课程《第五–六章：基于视觉与 AI 定位的多机协同控制实验》[1]，系统同时包含人形机器人搬运和多车跟随两项任务。TonyPi 从指定位置抱起目标物体，在障碍通道中至少完成两个转角（左右转各至少一次），到达终点后放下物体；TurtleBot2 保持跟随并在终点区域停靠。课程总体评价由现场竞赛与方案、源码报告构成，权重分别为 80% 和 20%。手册中的任务平台数量写为 3–4 台 TurtleBot2，详细评分表按四台、四种方式设置；本报告为四种方式保留独立章节，实际参赛车辆与方式认定须由全队和教师确认。

**表 1:** 任务要求与对应记录

| 要求 | 系统对应内容 | 应提交的证据 |
|---|---|---|
| 抱持、转弯与放置 | 抓取动作、上肢锁定、A 左转/B 右转/C 停止 | 连续视频、动作文件版本、决策日志 |
| 不接触障碍物 | 人形路线约束；TurtleBot 代价地图与门控 | 接触次数、障碍位置、最小间距实测 |
| 保持跟随目标 | 标记身份、目标时效检查、失效处置 | 丢失次数、持续时间、重捕获记录 |
| 多种独立方式 | 四组分别说明观测来源和控制链 | 各组实现、接口及现场运行证据 |
| 协同到达终点 | 任务状态、各车停靠条件 | 全队同场运行记录与到达时间 |

## 1.2 场地与感知约束

现场使用灰色地板及相对连续的灰色挡板构成通道，路线包含直行、左转、直行、右转和终点区域。地板接缝、胶带及其他地面标识均视为无关背景，不能作为路线识别标准。目标物块由早期蓝色调整为红色；抱持后物块可能遮挡画面下部，因此抓取确认与前方导航需要分别配置观察区域或头部姿态。

黄色雪糕筒可作为早期道路居中方案的终点候选；最新标记路线以固定标记 C 的距离触发停止。两种终点条件属于不同程序配置，不应在同一次实验记录中混写。图1仅表达路径拓扑，方向和长度以现场测量为准。

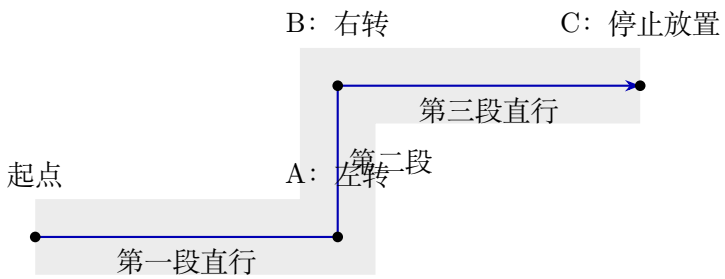



**图 1:** 路线拓扑示意；不按比例绘制，地面标识不参与定位

现场图片待归档
figures/site/corridor.jpg
选用通道全景；注明拍摄位置、日期和行进方向。

**图 2:** 灰色挡板通道现场（原始图片待归档）

# 2 团队组织与成员分工

## 2.1 小组之间的职责

第一组负责本次撰写的人形机器人与方式二部分。其余采用“第 $i$ 组”代称；下表为方便协作，暂将第二、三、四组分别对应方式一、三、四，需由对应小组确认后定稿。仓库部分历史 README 使用“本组负责方式三”的叙述，不能据此推断第一组成员身份。

**表 2:** 全队功能分工（人员信息待核实）

| 功能组 | 工作范围与交付 | 协同接口/负责人 |
|---|---|---|
| 第一组：人形机器人 | 动作录制与组合、抓取/放置、相机调试、视觉闭环路线、运行日志 | 目标背部标记、任务状态；姓名待填 |
| 第二组：方式一（暂定） | AR 直接跟随实际代码、标签尺寸、速度控制、失去标记处置；核对与课程固定航路点方式的区别 | `follow1`；姓名待填 |
| 第一组：方式二 | 地图采集、AMCL、AR 转地图、轨迹目标、move_base 与速度门控 | `follow2`；姓名待填 |
| 第三组：方式三（暂定） | RGB-D 目标检测、身份关联、实时避障与独立验收 | `follow3`；姓名待填 |

| 功能组 | 工作范围与交付 | 协同接口/负责人 |
|---|---|---|
| 第四组：方式四（暂定） | 前一台 TurtleBot 位姿/轨迹获取、队列间距与末车停靠 | `follow4`；姓名待填 |
| 联调与报告 | 网络/TF 核对、各方式独立性确认、数据汇总、视频与报告合并 | 总负责人、各组对接人待填 |

## 2.2 本小组内部职责登记

目前未提供成员姓名及个人贡献，以下是可认领的工作项，**不是对实际个人贡献的认定**。一人可承担多项，一项可由多人共同完成；贡献比例仅在成员协商后填写。

**表 3:** 本小组成员与贡献登记

| 姓名/学号 | 职责方向 | 具体工作及交付证据 | 贡献说明/确认 |
|---|---|---|---|
| 待填写 | 动作与机体 | 录制抓取/放置，组合抱物步态，检查姿态衔接；对应动作文件与提交号 | 待成员核实 |
| 待填写 | 视觉与状态机 | 相机维护、标记检测、路线切换、窗口与日志；对应代码与运行记录 | 待成员核实 |
| 待填写 | 建图与导航 | Kinect/扫描调试、建图、AMCL、代价地图与 DWA 参数；地图及配置 | 待成员核实 |
| 待填写 | 跟随与协同 | AR 地图目标、轨迹回退、速度门控、网络及任务状态；联调日志 | 待成员核实 |
| 待填写 | 测试与报告 | 数据采集、统计、图片归档、章节编写与排版；原始记录及文稿 | 待成员核实 |

## 2.3 其他小组内部职责登记规范

方式一、三、四的章节各预留一张成员分工表。各组填写“姓名、学号、具体任务、交付证据、合作边界”，避免只写“参与调试”或“负责报告”。最终由各组负责人核对，

并在全队分工表登记对接人。

# 3 系统架构与协同接口

## 3.1 分层控制

人形机器人负责物体搬运与领行；TurtleBot 负责各自的目标观测、定位和运动控制。不同机器人的执行器由各自进程独占，感知线程只提供观测，避免多个程序同时写舵机或底盘速度。图3描述方式二的完整数据链；任务心跳是可选适配接口，不代表所有人形入口已经发送心跳。

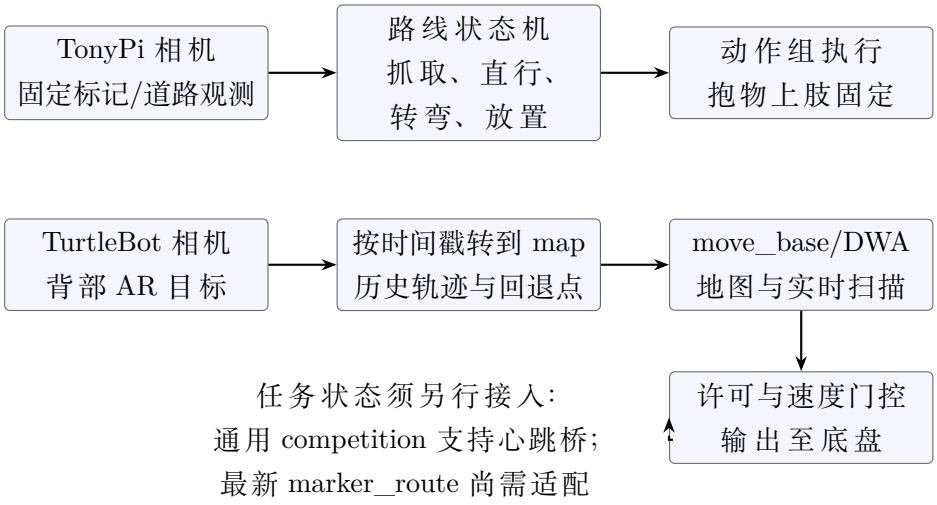



**图 3:** 人形机器人与方式二跟随的数据链

## 3.2 软件平台与版本边界

TonyPi 使用 Python 3、OpenCV、NumPy、PyYAML 及设备原装舵机 SDK；不同树莓派镜像的串口、SDK 和相机接口存在差异，应使用实际部署配置。TurtleBot 课程虚拟机为 Ubuntu 16.04、ROS Kinetic、Python 2.7，功能包放置于 `~/catkin_ws/src`，不能直接用 Python 3 替换 ROS 节点解释器。

## 3.3 拓扑与跟随方式的区分

“TonyPi 到首车再到后车”的队列拓扑，与“标记/地图/RGB-D/前车轨迹”的跟随算法是两个维度。当前方式二默认直接观察 TonyPi 背部 AR 标记，再在地图中导航；也支持输入前车地图位姿。实验时应固定一个目标来源并记录，不能把两个适配器同时发布到同一个目标话题。

**表 4:** 关键接口及约束

| 接口 | 内容 | 使用约束 |
|---|---|---|
| /ar_pose_marker | 带 ID 与时间戳的标记位姿 | ALVAR 标记；尺寸、相机内参和坐标系须一致 |
| target_pose | 目标在地图中的位姿 | 按观测时刻变换，保留原时间戳 |
| /scan | 障碍扫描 | 实际可能来自 Kinect 深度转扫描；检查高度覆盖与时效 |
| map->odom | 全局定位变换 | 建图用 GMapping；固定地图定位用 AMCL，不能同时发布 |
| /competition/mission | 搬运任务状态 | 需持续更新；开环和最新标记入口不能默认视为已接入 |
| /cmd_vel_mux/input/navi | 方式二底盘导航速度 | 由独立门控输出；不与键盘遥控争用 |

多机应使用唯一节点名、命名空间与 TF 名称，并确保时间与地图原点一致。虚拟机 NAT 地址可供宿主机 SSH 访问，不保证机器人能反向访问 ROS；联调需验证共同局域网、ROS_MASTER_URI 与ROS_IP，报告不收录密码等登录凭据。

# 4 第一组：TonyPi 人形机器人抱物与视觉路线控制

## 4.1 方案演进与入口区分

本部分的核心思路是保留人工录制且已经具备抱持效果的抓取动作，以动作组作为运动基本单元，将视觉观测作为动作选择与状态切换条件。机器人双足步态受负载、关节偏置及地面摩擦影响，固定动作次数容易累积航向误差，因此从开环演练逐步转向闭环控制。

## 4.2 动作组格式与抱物步态组合

.d6a 文件实质是 SQLite 数据库，ActionGroup 表按Index 顺序保存动作帧。每帧由帧序号、持续时间（毫秒）及各ServoN 位置刻度组成，历史文件存在 16 通道和 18 通道两种宽度。总线舵机刻度与头部 PWM 脉宽不是同一种单位，不应将“刻度 500 中位”套用到头部 1200 或 1500 的 PWM 参数。角度与位置的关系可写为

$$q_i = k_i(p_i - p_{0,i}), \tag{1}$$

**表 5:** 人形控制程序的用途与边界

| 入口 | 主要逻辑 | 本报告采用的地位 |
|---|---|---|
| `open_loop.py` | stand、grab、定次数前进/左右转、release | 调试基线，不能补偿累计偏移 |
| `competition.py` | 识别/抓取确认、固定或视觉路线、放置确认；支持人工 stand 到 grab 起步 | 综合反馈框架，需现场 ROI 与任务接口配置 |
| `lane_trial.py` | 人工摆位后 stand 到 grab，有限次数道路居中/纠偏 | 局部闭环试验，不包含完整两次路口任务 |
| `maze_route/marker_route.py` | 抱起后依次靠近 A、左转找 B、右转找 C、停止放置 | 最新独立路线实现；局部实机验证，完整路线待验收 |

其中 $p_i$ 为位置刻度，$p_{0,i}$ 为约定参考刻度，$k_i$ 包含方向和角度比例。刻度 500 只是常用中位参考，不能单凭此值认定机械关节角为零；本方案主要复用实录动作，不依赖完整关节绝对角标定。

令抓取动作最后一帧的上肢集合为 $\mathcal{A} = \{6, 7, 8, 14, 15, 16\}$，步态第 $k$ 帧各舵机位置为 $p_{k,i}^{\text{gait}}$，则抱物组合动作定义为

$$p_{k,i}^{\text{carry}} = \begin{cases} p_{\text{last},i}^{\text{grab}}, & i \in \mathcal{A}, \\ p_{k,i}^{\text{gait}}, & i \notin \mathcal{A}. \end{cases} \tag{2}$$

这一组合保持下肢帧序列、持续时间与其余通道不变，只替换六个手臂通道，减少转向步态破坏夹持姿态的可能性。历史版本分别组合过`go_forward`、`go_forward_fast` 及快速转向动作；最新标记路线使用`grab_final_v2` 和`catch_ball_*`，不能跨版本混用抓取末帧。

最新动作副本中的六个抱持刻度依次为 566、908、40、435、108、935。`mkcopies.py` 从设备原始动作生成独立副本，并逐帧核对未修改的腿部、索引和时间。放置动作仅在第一帧下蹲时保持抱紧手臂，后续降下、松开及站起帧保留原动作。运行时若首帧与当前姿态差值超过 25 刻度，将过渡时长延长到 300 ms；这一过渡不等于动态稳定性已获证明。

针对机体前倾，仓库另存两组大腿舵机相反方向的 20 刻度偏置候选，均带`_offset` 标记。该方案未作为当前机器人的默认步态，也没有完整实机稳定性结论，不能在报告中写成最终校正量。

## 4.3 抓取、持物观察与头部姿态

早期通用视觉框架采用 HSV/LAB 分割、形态学去噪及轮廓筛选；物块改为红色后使用跨越 HSV 色相两端的双阈值。通用框架可在抓取后检查持物 ROI，失败时经复位再识别，并在放置后检查手中区域与地面区域。人工起步模式则从 stand 直接衔接 grab，省去搜索与接近阶段。

最新`marker_route` 默认原地播放`grab_final_v2` 后锁定上肢，**并未接入上述颜色持物确认、自动失败重抓和放置结果确认链**。因此“抓取动作完成”与“物块已可靠抓住”应分别记录，不能将通用框架的功能当作最新入口已验证的能力。比赛要求掉落后回到掉落点重新抱起；当前路线程序也没有完成该恢复闭环。

头部参数按程序和目标分别标定：早期道路/持物调试使用 PWM servo1=1200，servo2 保持当前值；最新标记路线默认 pitch=1500、pan=1482，用来观察前方墙面标记。不同俯仰角会改变地面可见区域、物块遮挡与近距离码出画面的位置，报告应为每次运行登记真实参数，不能把两套姿态并写成同一配置。

## 4.4 早期灰色挡板道路居中

地面与挡板都为灰色，仅靠颜色阈值难以区分。早期道路算法使用边缘、线段、挡板支撑立柱与底边的几何关系，排除孤立地缝和地面胶带。抱持物块所在区域单独遮罩，避免把红色物块边缘误判为道路边界。

在近、远两个图像截面分别得到左右边界 $x_L(y), x_R(y)$，可定义归一化横向误差与方向误差

$$e_c = \frac{x_L(y_n) + x_R(y_n)}{2W} - \frac{1}{2}, \tag{3}$$

$$e_h = \frac{x_L(y_f) + x_R(y_f) - x_L(y_n) - x_R(y_n)}{2W}. \tag{4}$$

其中 $W$ 为图像宽度。组合误差超过阈值时选择左/右纠偏动作，误差处于容差内时执行一次前进，等待动作结束后重新观察。观测需连续稳定，不能在一张模糊帧上立即切换。挡板同色、遮挡、低对比和双足运动模糊仍可能导致无效道路判断；该方法不提供米制障碍距离或通用绕障规划。

## 4.5 最新 A/B/C 固定标记识别与测距

最新程序不依赖地面标识，而是在墙面布置三张已知模板标记。每张标记由 $9 \times 9$ 格构成，外两格为黑边，中间 $5 \times 5$ 格为数据，黑色正方形边长 14 cm。检测过程为自适应阈值、四边形轮廓筛选、透视展开、单元格采样，以及四个旋转方向的模板匹配；允许的数据位误差为 1 格。它们是程序自定义模板，**不能当作 ArUco、AprilTag 或 TurtleBot 所用 ALVAR 标签互换**。

以标记四角为三维已知点，结合相机内参 $K$ 与畸变参数，通过 PnP 求取旋转 $R$ 和平移 $t$：

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} R \mid t \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}. \tag{5}$$

检测模块输出摄像头到标记中心的直线距离、方位角及墙面朝向。内参读取设备上的相机标定文件。码中心与镜头存在高度差时，PnP 直线距离不能直接等同于沿地面水平距离，转角距离需用真实摆位复核。

## 4.6 完整路线状态机与自适应纠偏

路线状态依次为准备、抱起、走向 A、左转寻找 B、走向 B、右转寻找 C、走向 C、停止与放置。距离阈值默认 40 cm；路口并非固定播放若干次就认定 90 度转弯，而是沿预定方向逐次转动，直到下一张目标码进入视野，再由直行阶段继续对中。每次转前后分别等待 300 ms，每次前进后等待 200 ms。

为降低步态大转角引起的左右摆动，程序对连续 3 帧观测取中位数，并根据近期单次转向后的方位变化估计转角 $\hat{\theta}_{\text{step}}$。纠偏门限采用

$$|\beta| > \max\left(8^\circ, 0.6\hat{\theta}_{\text{step}}\right), \tag{6}$$

其中 $\beta$ 为目标方位角。程序还从可见标记时的相邻步观测估计直行偏航，并在目标暂不可见时进行预测补偿。**标记不可见与相机无新帧是不同故障**：前者在计数约束下仍可能继续走，后者等待恢复。默认未见目标的盲行上限为 25 步，目标曾可见后连续丢失也有暂停阈值；这一实现与课程“丢失后原地重捕获”的严格要求存在差距，需在正式联调前收紧失效策略并验收。

当标记在近目标区域居中消失时，程序可使用最近观测判定已到达。这是处理近距离码从画面下缘离开的启发式，不能替代障碍测距。路口最多尝试 12 次转向；连续 20 步推进不足 5 cm、总步数过多或相机长时间无图像时触发暂停/退出。默认没有深度或测距输入，因此尚不能声称具备完整防碰撞保证。

## 4.7 相机可靠性与可视化调试

现场曾出现`/dev/video0` 消失，内核提示 USB 枚举错误和线缆异常，重新插接后恢复。早期通用框架通过本地 HTTP 快照共享网页相机，避免重复打开已被占用的设备；最新`robust_camera.py` 则按 USB 身份查找相机、处理网页相机释放和断线重连，只返回 0.5 秒内的新画面。两种相机管理模式应各自独立运行。

最新窗口显示运行阶段、目标标记框、标记 ID、距离与方位等信息；支持 S 启动/恢复、P 暂停、Q 或 Esc 退出。每次运行保存`route.log`、`decisions.csv` 及标注截图，用于核对状态转移的观测依据。截图中识别框和数值只证明该时刻的检测，不证明全程搬运成功。

现场图片待归档
`figures/humanoid/marker_debug.jpg`
选用已提供的屏幕照片原件，保留标记框和日志；避免以照片读数替代日志统计。

**图 4:** 标记航路的实机窗口（原始图片待归档）

## 4.8 已有验证与后续验收

最新仓库说明记录实机已跑通“抱起、走向 A、在约 39.6 cm 处开始左转”；完整 A/B/C 路线仍未实机验收 [3]。仓库测试使用合成透视图验证标记识别与 PnP，并用给定步长、偏航和转角模型演练路线；合成实验精度与路线完成不能作为实地成功率。

后续应重点记录：满载单步距离与偏航、左右转角分布、两次路口完成率、物块掉落/接触次数、相机断线频次及完整路线时长。抓取成功确认、丢码原地重捕获、掉落恢复、终点放置确认以及任务心跳适配，仍需在最新标记入口中形成一致闭环。

# 5 第二组（暂定）：方式一 AR 直接跟随

## 5.1 已有实现摘要与方式认定

仓库已合并从虚拟机提取的`ar_follower` 包及独立实验说明 [5]。该实现通过 Kinect 检测前方机器人背部 ALVAR 标记，取标记光学坐标中的前向距离 $z$ 和横向偏差 $x$，以比例控制输出速度：

$$v = \mathrm{sat}(K_v(z - d_0)), \qquad \omega = \mathrm{sat}(-K_\omega x). \tag{7}$$

实际符号以节点坐标约定为准。默认目标 ID 为 1、跟随距离 0.4 m、最大线速度 0.3 m/s、最大角速度 1.0 rad/s。前方 60 度扇区障碍小于 0.25 m 时停止前进并慢速左转。该节点直接输出到`/cmd_vel_mux/input/teleop`，不经过方式二的速度门控。

课程参考的方式一强调固定场地标签航路点，而该组已提交程序属于移动目标背贴 AR 直接跟随，两者存在差别。应由对应组明确最终参赛定义、与方式二/三的独立性及教师确认情况，不在本初稿中强行等同。

## 5.2 本组应补充的内容

**待补充：**填写实际组号、现场标签尺寸、距离标定、完整跟随记录、转角与遮挡试验。现有节点缺少扫描/目标消息超时看门狗，不能将收到空检测消息时停车等同于所有通信中断时停车。

| 姓名/学号 | 具体任务 | 交付证据/合作说明 |
|---|---|---|
| 待填写 | 标记与相机标定 | 待填写 |
| 待填写 | 跟随控制与实机测试 | 待填写 |
| 待填写 | 数据整理与报告 | 待填写 |

**表 6:** 第二组（暂定）内部职责

# 6 第一组：方式二建图避障跟随

## 6.1 方案与功能包组织

方式二将“看见目标”与“安全到达跟随位置”分开：相机观测 TonyPi 背部 AR 标记，在地图中恢复其位置并积累历史轨迹；随后由导航栈在已知地图和实时障碍约束下生成底盘速度。目标朝向不直接作为跟随方向，避免背贴标签朝向与实际行进方向不一致时生成错误目标。

独立 ROS 功能包`multi_robot_following` 位于`turtlebot/follow2/`，部署到虚拟机`~/catkin_ws/src`。包内包括建图入口、AMCL/move_base 配置、目标源适配、轨迹跟随、速度门控与任务心跳桥，不覆盖原有课程包。

## 6.2 沿用课程建图链与地图质量检查

底盘启动与键盘遥控已由小组确认可用：`turtlebot_bringupminimal.launch`及`turtlebot_teleopkeyboard_teleop.launch`。建图优先沿用课程`turtlebot_navigationgmapping_demo.launch`，另用课程导航 RViz 配置查看地图、扫描和 TF。该入口已集成部分相机和导航节点，不与新包的建图入口重复启动。

建图时采集里程计和扫描，通过 GMapping 估计二维栅格地图；完成后使用`map_saver` 成对保存`.yaml` 与`.pgm`。比赛导航加载固定地图，AMCL 估计当前车位姿，move_base 进行规划。场地障碍布局改变需重新建图；从建图切换到定位时，必须停止 GMapping，避免与 AMCL 竞争`map->odom`。

现场曾报告约 35 cm 高障碍未在图中稳定出现。不能只按 URDF 假定镜头高度便

认定扫描必然覆盖：应分别检查原始深度图、深度转扫描的采样行与范围、实际安装俯仰、TF、有效近距，以及 RViz 中/scan 是否已有对应点。若扫描本身漏检，调节地图膨胀层不能补回未观测障碍；若扫描有点而地图无障碍，再排查建图、TF 和清除配置。当前尚未提供漏检已解决的完整实测记录。

现场图片待归档
figures/follow2/robot.jpg
归档已提供的机器人照片；补充镜头高度、朝向、扫描来源。

**图 5:** TurtleBot2 与 Kinect 安装（原始图片待归档）

## 6.3 AR 目标观测到地图坐标

方式二默认target_source:=marker，选取指定 ID 的 ALVAR 标记。marker_target.py 验证时间戳、有限数值与量测范围，在观测时刻 $t_m$ 将目标变换到地图：

$$ {}^{m}T_{\text{target}}(t_m) = {}^{m}T_{c}(t_m)\,{}^{c}T_{\text{target}}(t_m). \tag{8} $$

其中 $c$ 为相机观测坐标系，$m$ 为地图坐标系。输出保留原始量测时间，TF 缺失、过期或外推失败时丢弃观测，不以当前时间重新包装旧数据。节点默认量测时效 0.5 s、最大量测范围 4 m，这些是代码限制，不是实际检测精度。

另一适配器可订阅前车地图定位位姿，但必须共享地图原点并检查时间戳、跳变和协方差。当前第一组方案以直接观察 TonyPi 为主；只有确认切换到前车定位模式时，才在实验表中登记该数据源。

## 6.4 历史轨迹与沿路径回退的跟随点

Trail 将有效目标位置记录为有界折线 $P = (p_0, \ldots, p_n)$。默认最小采样间距 0.08 m、最大保留路径长度 20 m、单次跳变上限 0.8 m。跟随距离默认 $d_f = 0.9\,\text{m}$，从最新点沿折线反向累计弧长；若回退点位于 $p_{j-1}$ 与 $p_j$ 之间，令该段长度 $\ell_j = \|p_j - p_{j-1}\|$、剩余距离 $r$，则

$$ p_g = p_j + \frac{r}{\ell_j}(p_{j-1} - p_j). \tag{9} $$

轨迹不足 $d_f$ 时不生成有效目标。与沿目标当前朝向直接平移相比，这种方法保留已观测路径的转角信息。但 move_base 仍会在可通行区域平滑或重规划，不能宣称精确重放前车全部轨迹，也不能保证后车永远不抄近路。

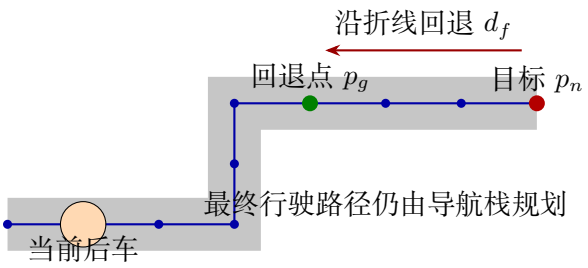



**图 6:** 沿目标历史折线回退生成导航目标；示意图不含真实场地尺度

## 6.5 代价地图、DWA 与目标可行性

全局规划依托固定地图及目标点；全局与局部代价地图接收实时扫描，DWA 在速度与加速度约束下评估短时轨迹。候选目标在发送前执行圆形足迹栅格检查，越界、未知或高占用栅格均不可进入。足迹、膨胀半径及障碍更新参数必须按真实车体和通道标定，默认值不是经过测量的保证。当前配置禁止倒车，并关闭自动原地旋转恢复，以降

**表 7:** 方式二部分默认参数（非最终实测值）

| 参数 | 默认值 | 作用 |
|---|---|---|
| 跟随路径间距 | 0.9 m | 与目标沿历史路径保持滞后 |
| 最大前进速度 | 0.15 m/s | 限制跟随底盘速度 |
| 最大角速度 | 0.5 rad/s | 限制转向速度 |
| 线加速度限制 | 0.3 m/s$^2$ | DWA 动态约束 |
| 控制/规划频率 | 10/1 Hz | 局部控制与全局重规划 |
| DWA 预测时长 | 1.5 s | 轨迹评估窗口 |
| 平面目标容差 | 0.15 m | 到达导航目标的容差 |

低拥挤通道中盲目恢复的风险。其代价是某些受阻情形需要重新规划或人工复位，而不能自行摆脱。视野之外、深度缺测或低于有效扫描面的物体仍可能漏检，导航栈不等于全空间避碰。

## 6.6 双层许可与速度门控

跟随控制层检查目标、定位、TF、地图/代价地图、任务状态与目标最小间距，满足条件才发导航目标与运动许可。独立`velocity_gate.py` 进一步要求许可、导航速度和扫描均新鲜，检查前方扇区有效量测并拒绝倒车、NaN/Inf 及非法速度。可概括为

$$u_{\text{out}} = \begin{cases} \text{limit}(u_{\text{nav}}), & G_{\text{mission}} \wedge G_{\text{target}} \wedge G_{\text{loc}} \wedge G_{\text{map}} \wedge G_{\text{TF}} \wedge G_{\text{scan}} \wedge G_{\text{cmd}}, \\ 0, & \text{其他情况.} \end{cases} \tag{10}$$

其中 $G$ 表示相应许可；它们分别由上游与独立门控实现，并非一个节点完成所有检查。NaN/Inf 占满的扫描扇区被视为缺测，不当作空闲空间。门控具有壁钟检查线程，以避免 ROS 仿真时钟停顿后速度无法及时归零。

AMCL 在机器人静止时不一定持续发布位姿，程序定期请求无运动定位更新，并配置重采样频率，使新扫描可更新定位结果。请求本身不制造有效扫描或虚假新鲜度，定位故障仍需在 RViz 重新初始化或排查传感器。

## 6.7 任务状态联动与当前集成缺口

跟随器需要持续收到允许阶段的`/competition/mission`。通用`competition.py` 可通过 UDP 心跳桥提供状态；`open_loop.py` 不发送该心跳，最新`marker_route.py` 同样未实现该接口。因此“两个程序分别能启动”不等于“已经自动协同”：需增加适配器，将标记路线的搬运、放置、完成、暂停和故障映射到任务状态，明确频率、超时和停止行为后再联调。

单车导航调试时可以在终端持续发布`carrying`，但这仅是测试输入，不能写成全队自动联动已完成。终点还需核对后车是否真正进入终点区域并停稳，不能只用 TonyPi 的放置完成判定所有车辆到达。

## 6.8 验证范围与待测指标

已有部署记录表明功能包在 ROS Kinetic/Python 2.7 下完成构建、语法与 launch 解析检查；隔离 ROS master 中，在缺少目标、定位、任务与扫描时观测到零速度和禁止运动。该检查未向真实底盘发送导航速度，证明的是部分失效路径的逻辑，不能推导实际绕障成功率。

本组已确认底盘和键盘遥控可用，地图质量、AMCL 稳定性、动态避障、目标丢失后原地恢复及全程跟随仍需按第9节采集真实数据。尤其应在 35 cm 障碍、转角目标遮挡和网络中断条件下逐项复核，而非直接进行多车混合测试。

# 7 第三组（暂定）：方式三 RGB-D 实时检测与避障跟随

## 7.1 已有实现摘要

仓库`turtlebot/follow3/rgbd_following` 已包含颜色与深度候选检测、多帧目标关联、外观模板及有界重识别、历史轨迹目标、滚动里程计代价地图与独立速度门控 [6]。其观测来源为实时 RGB-D 与本车里程计，不依赖预建地图或前车广播位置，区别于方式二的固定地图导航。

现有说明将检测阈值作为示例，要求登记真实身份模板并在独立样本上评估。区域深度有效不等于识别到了 TonyPi；颜色候选也不能替代身份连续性验证。源码和合成演练已提供，真实目标识别、导航与全队终点验收仍需对应组确认。

## 7.2 本组补充区域

**待补充：**实际组号、识别特征与数据集、RGB/深度配准、遮挡重识别策略、主动绕障方式、默认参数与实测结果。应给出误识别、目标切换、目标丢失及终点停车的原始证据。

| 姓名/学号 | 具体任务 | 交付证据/合作说明 |
|---|---|---|
| 待填写 | 数据与目标识别 | 待填写 |
| 待填写 | 控制、避障与联调 | 待填写 |
| 待填写 | 评估与报告 | 待填写 |

**表 8:** 第三组（暂定）内部职责

# 8 方式四：TurtleBot2 视觉跟随与队列行驶

## 8.1 方案与队列关系

竞赛要求 TurtleBot2 自主跟随目标机器人，跟随过程中不得碰撞障碍物或其他机器人，也不得丢失目标。本组使用三台 TurtleBot2 纵列跟随：一号车以点云视觉跟踪 TonyPi，二号车跟随一号车，三号车跟随二号车。二、三号车通过 Kinect 识别前车背部的 AR 码标签，按标签相对位置控制底盘；深度图转换得到的扫描数据用于近距离障碍响应。该队列方案不使用地图定位和全局路径规划。

一号车和二号车尾部分别粘贴 ID 0 和 ID 1 标签，三号车尾部未贴标签。二号车识别一号车的 ID 0 标签，三号车识别二号车的 ID 1 标签。一号车通过点云数据识别并跟踪 TonyPi，不使用 AR 标签。以下功能包和启动参数说明适用于二、三号车。

## 8.2 系统组成与控制方法

功能包`team_turtlebot_follow_plus`使用 ROS Kinetic/C++ 实现，位于`turtlebot/follow4/team_turtlebot_follow_plus/`。`ar_track_alvar`根据 Kinect RGB 图像和相机内参发布标签位姿；`depthimage_to_laserscan`将深度图转换为`/scan`。跟随

节点订阅/ar_pose_marker 与/scan，向/mobile_base/commands/velocity 发布速度指令。

设相机光学坐标系中的标签位置为 $(x, z)$，其中 $x$ 向图像右侧为正、$z$ 指向相机前方，则标签距离与方位角为

$$r = \sqrt{x^2 + z^2}, \qquad \theta = \operatorname{atan2}(x, z). \tag{11}$$

控制节点以 10 Hz 运行，按距离误差比例调节前进速度，按方位角调节转向速度；线速度限制在零至设定上限之间，目标距离不足时不倒车，标签偏离车头方向较大时暂停前进。

避障逻辑将扫描数据划分为前方及左右侧扇区。前方出现近障时，节点选择有足够扫描净空的一侧并保持该转向选择；距离达到停车阈值或所选侧净空不足时停止前进。扫描数据超时则发布零速度。目标首次出现前车辆保持停止；已识别目标丢失后，车辆低速原地搜索，超过设定时限仍未重获目标则停止。该方法依赖标签保持在相机视野内，不能沿被遮挡路线推断前车位置。

## 8.3 参数设置及含义

二、三号车使用相同的跟随和避障参数，目标标签 ID 分别设为 0 和 1。表9列出实际启动值及参数用途。

**表 9:** 二、三号车运行参数及含义

| 参数 | 单位 | 二号车 / 三号车 | 含义 |
|---|---|---|---|
| marker_id | – | 0 / 1 | 目标标签编号。节点只跟随该 ID 对应的标签。 |
| marker_size | cm | 14.0 / 14.0 | 标签黑色方框的实际边长，用于 AR 距离估计，应与打印尺寸一致。 |
| desired_distance | m | 0.40 / 0.40 | Kinect 到标签的目标距离，不等同于两车外壳间距。程序不倒车；标签近于设定距离时停止前进。 |
| max_linear_speed | m/s | 0.35 / 0.35 | 最大前进速度。实际速度随距离误差及避障控制变化。初次低速测试设为 0.05–0.10 m/s。 |
| max_angular_speed | rad/s | 0.25 / 0.25 | 正常跟随及近障转向的角速度上限。 |
| avoid_distance | m | 0.38 / 0.38 | 前方障碍进入该距离后，开始减速并向扫描较空的一侧偏转。 |
| stop_distance | m | 0.20 / 0.20 | 前方障碍达到该距离后，线速度置零；侧向净空足够时向较空的一侧转向。 |
| side_clearance | m | 0.40 / 0.40 | 转向侧所需的最小扫描净空。两侧均不足时车辆停止。 |

**表 9:** 二、三号车运行参数及含义（续）

| 参数 | 单位 | 二号车 / 三号车 | 含义 |
|---|---|---|---|
| avoid_release_margin | m | 0.20 / 0.20 | 障碍距离超过avoid_distance 加该余量后，退出避障状态并恢复跟随。 |
| obstacle_turn_speed | rad/s | 0.20 / 0.20 | 达到停车距离后，车辆向选定空旷侧转向时使用的角速度上限。 |
| marker_timeout | s | 0.8 / 0.8 | 标签超过该时长没有新检测结果后，节点开始目标搜索。 |
| lost_search_timeout | s | 16 / 16 | 原地搜索目标的最长时间；超时仍未重获目标时停止。 |
| search_angular_speed | rad/s | 0.20 / 0.20 | 目标丢失后原地搜索时的角速度，且不超过max_angular_speed。 |

## 8.4 执行流程与运行参数

将功能包放入~/catkin_ws/src/ 后编译：

```
source /opt/ros/kinetic/setup.bash
cd ~/catkin_ws
catkin_make
source devel/setup.bash
```

后车分终端启动底盘、Kinect 和深度扫描转换：

```
# TurtleBot2底盘
source /opt/ros/kinetic/setup.bash
source ~/catkin_ws/devel/setup.bash
roslaunch turtlebot_bringup minimal.launch

# Kinect
source /opt/ros/kinetic/setup.bash
source ~/catkin_ws/devel/setup.bash
roslaunch openni_launch openni.launch

# 深度图转换为 /scan
source /opt/ros/kinetic/setup.bash
rosrun depthimage_to_laserscan depthimage_to_laserscan \
  image:=/camera/depth/image_raw \
  camera_info:=/camera/depth/camera_info
```

确认/scan 持续更新后，二号和三号车分别使用下列参数启动跟随节点：

```
# 二号车跟随一号车（标签ID 0）
roslaunch team_turtlebot_follow_plus follow_ar.launch \
  marker_id:=0 marker_size:=14.0 desired_distance:=0.40 \
  max_linear_speed:=0.35 max_angular_speed:=0.25 \
  avoid_distance:=0.38 stop_distance:=0.20 side_clearance:=0.40 \
  avoid_release_margin:=0.20 obstacle_turn_speed:=0.20 \
  marker_timeout:=0.8 lost_search_timeout:=16 search_angular_speed:=0.20

# 三号车跟随二号车（标签ID 1）
roslaunch team_turtlebot_follow_plus follow_ar.launch \
  marker_id:=1 marker_size:=14.0 desired_distance:=0.40 \
  max_linear_speed:=0.35 max_angular_speed:=0.25 \
  avoid_distance:=0.38 stop_distance:=0.20 side_clearance:=0.40 \
  avoid_release_margin:=0.20 obstacle_turn_speed:=0.20 \
  marker_timeout:=0.8 lost_search_timeout:=16 search_angular_speed:=0.20
```

## 8.5 实验记录

三车联调结果汇总见表10。本次测试未专门布置障碍物。

**表 10:** 方式四实验记录

| 测试项目 | 实际情况 | 结果 |
|---|---|---|
| 直行跟随 | 队列间距保持稳定，速度适中，车辆间未碰撞 | 完成 |
| 转弯跟随 | 采用圆弧轨迹转弯，保持后车对前车标签的观测 | 完成 |
| 障碍响应 | 最终场地未专门布置障碍；未碰撞围栏，但围栏及长时间停留的前车曾触发误避障 | 有待优化 |
| 目标保持 | 本次联调未发生目标丢失，重捕获逻辑未实际触发 | 未覆盖 |
| 终点停车 | 三车依次到达终点并平稳停靠 | 完成 |

赛道围栏未发生碰撞，但当前车低速行驶时，后车曾把围栏识别为障碍并触发避障；当前车长时间停留在近处时，后车也曾将其判断为障碍，继而原地转向。后续需结合赛道宽度和前后车间距调整避障距离及状态切换条件，并在围栏邻近、前车慢行和停车场景下复测。测试期间目标未丢失，目标重捕获功能尚无实际触发记录。

## 8.6 小组分工

**表 11:** 方式四小组分工

| 姓名/学号 | 工作内容 | 交付记录 |
| --- | --- | --- |
| 徐杰鸿 24363164 | 动态跟踪程序编写、避障功能优化及参数调试、跟随控制与障碍测试、报告整理 | 实地测试、参数调优及结果记录 |
| 王振宇 24363096 | 标签配置与安装，建立标签与车辆的对应关系，完成标签识别验证 | 标签 ID 清单、安装位置及车辆对应关系记录、识别测试结果 |
| 林琢严 24363054 | 辅助代码优化、场地布置 | 实验结果记录 |
| 何爽 24363028 | 辅助代码优化、场地布置 | 实验结果记录 |

## 8.7 小结

本组以点云视觉跟踪和 AR 码标签跟随组成三车队列：一号车跟踪 TonyPi，二号车识别 ID 0，三号车识别 ID 1；标签边长为 14 cm。联调中三车完成直行、圆弧转弯和依次停车，车辆未发生碰撞或目标丢失。围栏和慢行前车引发的误避障仍需通过参数调整和复测处理；本次未进行专门障碍布置，不能据此判定复杂障碍下的避障性能。

# 9 实验设计、证据与结果记录

## 9.1 验证层级与当前事实

本报告不将“已编写”“离线通过”“局部实机”和“完整验收”混为一谈。测试数量随仓库更新会变化，自动化测试通过也不能证明实际无碰撞。正式结果须绑定代码提交、设备、参数、场地与原始证据。

**表 12:** 已有证据及其范围

| 项目 | 当前证据 | 能说明/不能说明 |
| --- | --- | --- |
| 动作副本 | 数据库帧、时间与非手臂通道核对；独立副本生成脚本 | 说明组合遵循指定规则；不证明抱持稳定性 |
| 人形局部路线 | 最新 README 记录抱起、走到 A 并约 39.6 cm 处左转 | 支持首段局部运行；不证明 B/C 路线完成 |

| 项目 | 当前证据 | 能说明/不能说明 |
|---|---|---|
| 人形离线验证 | 合成标记透视图与给定步态模型路线演练 | 说明设定条件下算法逻辑；不是真实误差分布 |
| 相机链 | 历史恢复检查连续取得 15 帧 $640 \times 480$ 新图像 | 说明当次取流恢复；不保证持续运动不掉线 |
| TurtleBot 基础 | 小组确认底盘启动与键盘遥控可用 | 说明基础驱动可用；不等于自主导航成功 |
| 方式二隔离检查 | Kinetic 构建、launch 解析、缺少必要输入时零速度 | 部分软件与停止逻辑；未完成实地绕障验收 |
| 完整比赛 | 未提供完整连续运行及统计记录 | 成功率、总时长、碰撞与掉落次数均待测 |

## 9.2 分阶段实验与异常注入

1. **单动作与视觉**。人工监督下分别测试站立、抓取、抱物前进、左右转和放置；固定相机姿态后测量标记距离/方位，统计多次误差与失效距离。

2. **人形单段与完整路线**。先测直行偏移，再测 A 左转/B 右转，最后无人工干预完成 A/B/C 全程。分别记录动作次数、转角偏差、接触、掉落及相机断线。

3. **地图和单车导航**。确认 35 cm 障碍进入原始深度与扫描，再检查地图。先静态目标导航、定位丢失和规划失败，再接入移动 AR 目标。

4. **目标与任务失效**。遮挡标签、停止目标消息、停止任务心跳、断开扫描或使 TF 过期，逐项测量停止延迟。人形最新程序的盲行与近距丢码启发式应单独评估。

5. **多车联调**。固定队列拓扑、共享地图与时间基准，避免速度控制源冲突；记录各车到达终点条件，不只记录领行机器人结束。

## 9.3 指标定义

距离标定用均方根误差 $\mathrm{RMSE}_d = \sqrt{N^{-1}\sum_i (\hat{d}_i - d_i)^2}$；$d_i$ 必须是与 PnP 输出同一定义的真实距离。跟随间距误差定义为 $e_f(t) = d(t) - d_0$，报告平均绝对误差与最大偏差，同时说明 $d$ 采用欧氏距离还是路径弧长。完整任务成功率定义为 $N_{\mathrm{success}}/N_{\mathrm{trials}}$，每次试验均需记录，不能剔除失败试跑而不说明。

对接触次数、掉落次数、目标丢失持续时间、最长无新帧时间、停止延迟和各车到达时间分别统计。转角是否达到路线要求应通过外部测量或可复核视频判定，不能把“看见下一张码”直接记为精确 90 度。

**表 13:** 正式结果汇总模板；破折号表示未测，不表示零

| 项目 | 样本数 | 结果 | 证据编号 |
|---|---|---|---|
| 标记距离误差 | 待测 | — | 待填写 |
| 满载单步距离/偏航 | 待测 | — | 待填写 |
| 人形完整路线成功率 | 待测 | — | 待填写 |
| 方式二完整跟随成功率 | 待测 | — | 待填写 |
| 35 cm 障碍检测与避让 | 待测 | — | 待填写 |
| 目标/任务失效停止延迟 | 待测 | — | 待填写 |
| 全队到达率/总时长 | 待测 | — | 待填写 |

## 9.4 数据归档与照片使用

运行记录模板保存在`latex/data/experiment_records.csv`。每行对应一次试验，保存代码版本、设备、运行参数、结果及证据路径；原始视频和 bag 较大时放在团队共享存储中，报告记录可访问的证据位置。不要将不同机器人的动作参数或不同版本的相机设置合并统计。

用户提供的通道、实机窗口及 TurtleBot 照片可作为场地和调试过程说明；应将原图归档到`figures/site`、`figures/humanoid` 和`figures/follow2` 并注明出处。当前源码支持图片尚未归档时显示明确占位框；占位框不是实验结果。终稿应补齐实际引用照片或移除相应图位。

# 10 问题分析与总结

第一组围绕人形抱物和地图避障跟随建立了可分步调试的实现。动作层通过固定抓取末帧上肢、复用原步态下肢实现抱物前进与转向；视觉层从开环次数控制发展到道路几何居中及固定标记航路，减轻直行偏航的累计影响。方式二将 AR 相对观测变换到地图，利用历史路径回退生成目标，再交由导航栈规划，并以任务、定位和扫描等有效性条件约束速度输出。

实验过程表明，影响系统的因素不仅是控制算法，还包括动作版本、抱持负载、相机姿态、USB 供电与连接、深度扫描覆盖、多机 TF 与网络。仅根据模型镜头高度推断

障碍可见、仅根据动作次数认定转角或仅根据单帧识别认定完整跟随成功，都会使报告结论超出证据。

当前主要未完成项为最新标记路线的完整实机验收、抓取/掉落/放置反馈的一致接入、严格的丢目标原地恢复、任务心跳适配、矮障碍检测复核及全队同场结果统计。后续应以日志和连续视频建立可复核记录，再填写性能结果和成员最终贡献。其余小组应在各自章节补足实际实现与方式独立性说明，共同完成终稿。

# 参考文献

[1] 课程实验手册. 第五–六章：基于视觉与 AI 定位的多机协同控制实验. 仓库根目录同名.docx 文件。

[2] 项目协作仓库. https://github.com/yky666/intelligent_robot_system. 本次代码核对基线：6b6db8c，后续修改见版本历史。

[3] TonyPi 标记航路实现及实验状态. humanoid/maze_route/README.md，marker_route.py，markers.py，robust_camera.py，tests/test_maze_route.py。

[4] 第一组地图跟随实现与部署记录. turtlebot/follow2/multi_robot_following/，docs/vm_follow2.md，docs/turtlebot_debug_commands.md。

[5] AR 直接跟随实验总结与提取功能包. docs/ar_marker_follow_experiment_summary.md，turtlebot/follow1/ar_follower/。

[6] RGB-D 实时跟随方案、状态与验收要求. turtlebot/follow3/README.md 及对应功能包。

# A 复现入口与协作清单

## A.1 源码入口索引

| 内容 | 仓库路径 |
|---|---|
| 最新人形路线 | humanoid/maze_route/marker_route.py |
| 检测与相机 | humanoid/maze_route/markers.py，robust_camera.py |
| 动作副本 | humanoid/maze_route/actions/；生成规则见同目录mkcopies.py |

| 内容 | 仓库路径 |
| --- | --- |
| 通用反馈与道路 | humanoid/scripts/competition.py，lane_control.py，lane_trial.py |
| 方式二入口 | turtlebot/follow2/multi_robot_following/launch/vm_follow2.launch |
| 目标与导航控制 | 该包scripts/marker_target.py、map_follower.py、velocity_gate.py |
| 轨迹与几何核心 | 该包src/competition_core/planning.py |
| 导航默认参数 | 该包config/move_base.yaml 及costmap_common.yaml |

## A.2 只查看最新标记检测

在具备相机、标定文件及显示环境的 TonyPi 上，从完整仓库运行：

```
cd ~/intelligent_robot_system
python3 humanoid/maze_route/marker_route.py --no-servos
```

--no-servos 不访问舵机串口并隐含 dry-run；单独--dry-run 仍可能设置头部，二者不能混淆。真实运动入口会关闭厂家上位机并停止相关服务，运行前核对同目录 README 与设备配置；本报告构建不会运行机器人控制脚本。

## A.3 方式二现场启动索引

底盘、相机、扫描与地图请按docs/turtlebot_debug_commands.md 逐终端启动，避免重复驱动。完成地图及 AMCL 初始位姿后，主要跟随入口为：

```
source /opt/ros/kinetic/setup.bash
source ~/catkin_ws/devel/setup.bash
roslaunch multi_robot_following vm_follow2.launch \
  map_file:=$HOME/catkin_ws/maps/competition.yaml \
  marker_id:=1 marker_size_cm:=9.0
```

标记尺寸 9.0 cm 是该 launch 默认值，必须按实际 ALVAR 黑边尺寸替换；与人形航路 14 cm 自定义标记无关。正式跟随还需要持续任务状态和各项许可，单独启动此命令不会自动形成全队协同。

## A.4 报告提交前核对

1. 各组确认组号、成员姓名学号及真实个人贡献，修改分工表和封面信息。
2. 补齐照片原件、地图/参数版本、每次试验记录与连续运行证据。
3. 将合成测试与实测结果分开，填入样本数、失败次数及实际验收结论。
4. 核对四种方式的独立性、任务状态适配及终点停靠定义。
5. 编译并检查 PDF 目录、公式、表格和图片；删除或补齐所有待填写项目。